DATA 405 Assignment 3

Rin Meng, 51940633

Question 1

(a) Quantile Function: The CDF of V is:

$$F_V(x) = \begin{cases} 1 - e^{-x^2}, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

To find the quantile function, solve for x in terms of u:

$$F_V(x) = u \Rightarrow 1 - e^{-x^2} = u$$
$$e^{-x^2} = 1 - u$$
$$-x^2 = \ln(1-u)$$
$$x = \sqrt{-\ln(1-u)}$$

Therefore, the quantile function $Q(u)$ for V is:

$$Q(u) = \sqrt{-\ln(1-u)}$$

Then, we can write the `rmyV` function using the quantile function:

```
rmyV <- function(n) {
  u <- runif(n)
  x <- sqrt(-log(1 - u))
  return(x)
}
```

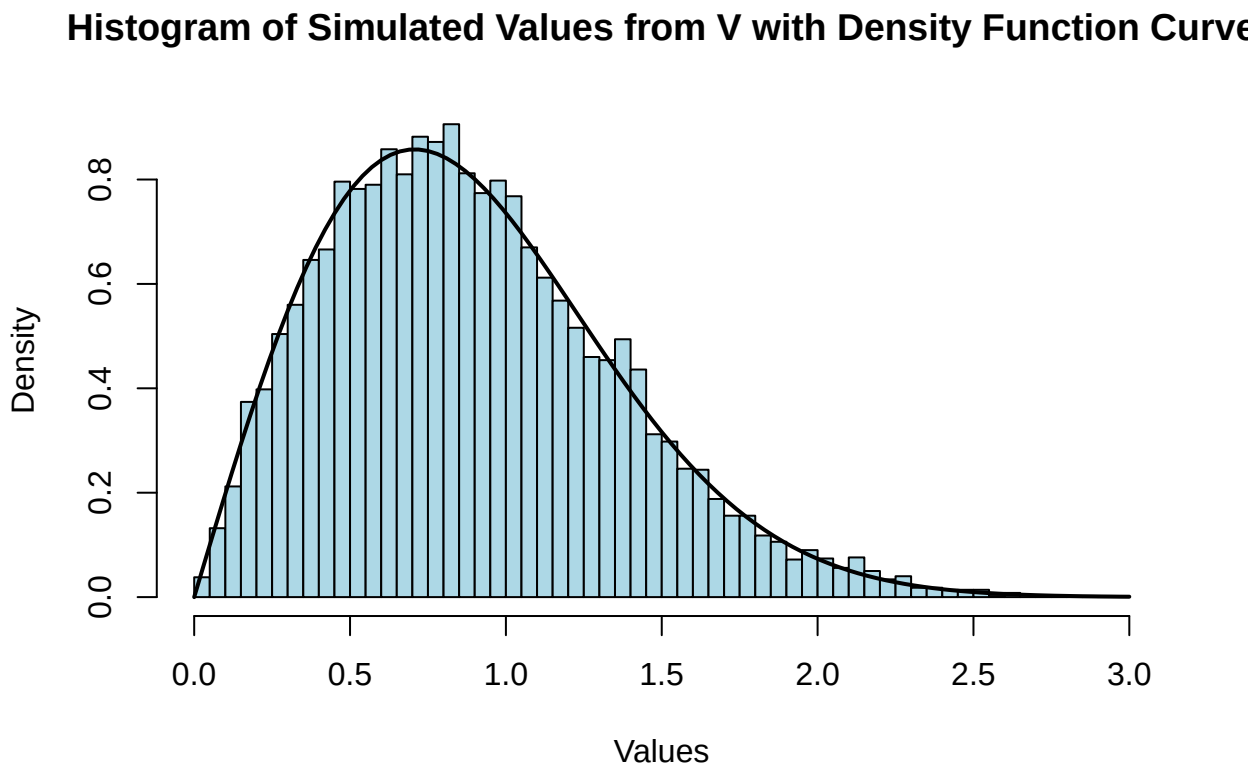
(b) The density function can be found by differentiating $F_v(x)$,

$$f_v(x) = \frac{d}{dx}F_V(x) = \frac{d}{dx}(1 - e^{-x^2}) = 2xe^{-x^2}, x > 0$$

Then we can start coding for the simulation and plotting:

```
dV <- function(x){
  ifelse(x > 0, 2 * x * exp(-x^2), 0)
}

set.seed(51940633)
```

```r
simVals <- rmyV(10000)

hist(simVals, probability = TRUE, breaks = 50,
main = "Histogram of Simulated Values from V with Density Function Curve",
xlab = "Values", col = "lightblue", border = "black")

curve(dV(x), add = TRUE, col = "black", lwd = 2)
```



Question 2

(a) The cumulative distribution of the function X can be written as:

$$F_X(x) = \int f_X(x)\,dx = \int 3x^2\,dx = x^3$$

So the CDF for $F_X(x)$ is:

$$F_X(x) = \begin{cases} x^3, & x \in [0,1] \\ 0, & x < 0 \\ 1, & x > 1 \end{cases}$$

(b) Now we can write the quantile function $Q(u)$, found by solving $F_X(x) = u$ for x, where u is a uniform random variable on $[0,1]$.

$$F_X(x) = u \Rightarrow x^3 = u$$

$$x = u^{1/3}$$

So then the quantile function $Q(u)$ is,

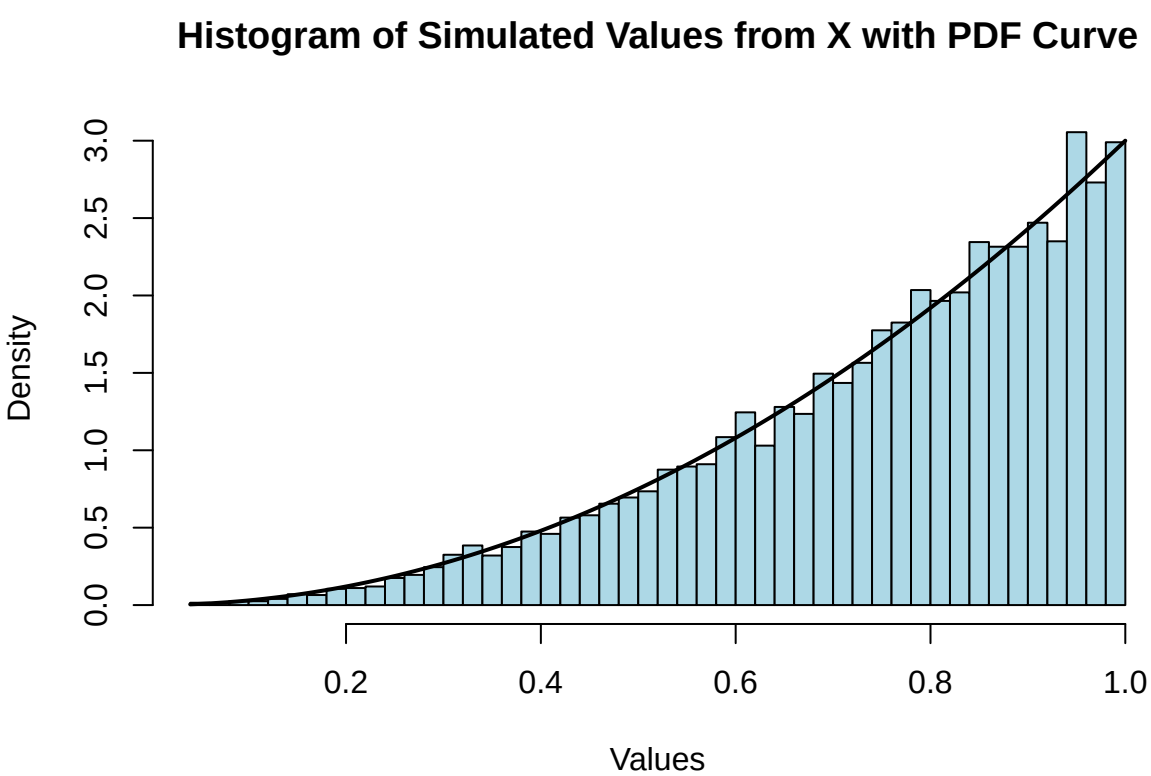
$$Q(u) = u^{1/3}$$

Now we can write the R function to generate random variates:

```
rmyX <- function(n){
  u <- runif(n)
  x <- u^(1/3)
  return(x)
}
```

Now we define the PDF function for x, and simulate the distribution:

```
dX <- function(x){
  ifelse(x >= 0 & x <= 1, 3 * x^2, 0)
}

set.seed(51940633)
simVals <- rmyX(10000)

hist(simVals, probability = TRUE, breaks = 50,
main = "Histogram of Simulated Values from X with PDF Curve",
xlab = "Values", col = "lightblue", border = "black")

curve(dX(x), add = TRUE, col = "black", lwd = 2)
```



Question 3

(a) The CDF of Y is given as:

$$G(y) = pF_V(y) + (1-p)F_X(y)$$

Where $F_v(y) = 1 - e^{-y^2}$ is from question 1 and

$$F_X(y) = \begin{cases} y^3, & x \in [0,1] \\ 0, & y < 0 \\ 1, & y > 1 \end{cases}$$

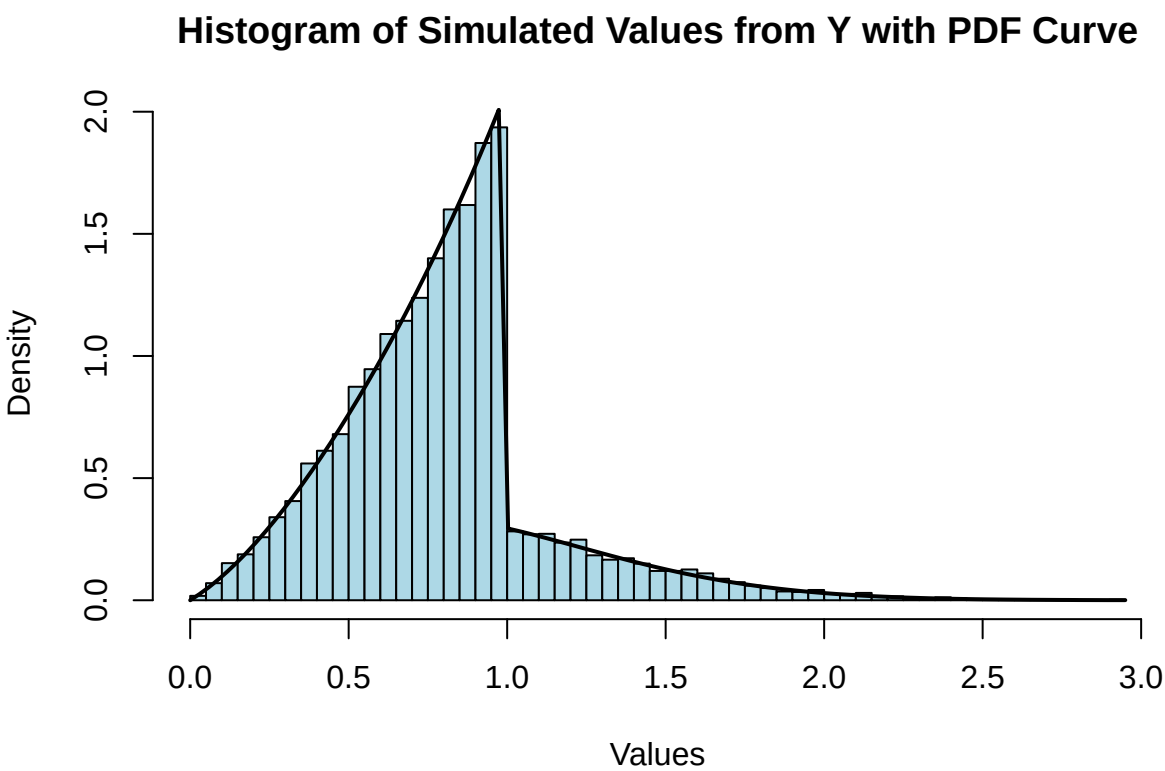
from question 2.

(b) We can now generate random variates from Y,

```
rmyY <- function(n, p) {
  # Step 1: Determine which distribution to sample from
  from_V <- rbinom(n, 1, p)  # Vector of 0s and 1s, where 1 means sample from V

  # Step 2: Generate samples from the appropriate distribution
  samples <- numeric(n)
  samples[from_V == 1] <- rmyV(sum(from_V == 1))  # Generate from V where from_V == 1
  samples[from_V == 0] <- rmyX(sum(from_V == 0))  # Generate from X where from_V == 0

  return(samples)
}
```

(c) Simulate the distribution of Y, for cases where p = 0.4:

```
gY <- function(y, p) {
  p * (2 * y * exp(-y^2)) + (1 - p) * (ifelse(y >= 0 & y <= 1, 3 * y^2, 0))
}

set.seed(51940633)  # for reproducibility
simulated_values_Y <- rmyY(10000, p = 0.4)

hist(simulated_values_Y, probability = TRUE, breaks = 50,
     main = "Histogram of Simulated Values from Y with PDF Curve",
     xlab = "Values", col = "lightblue", border = "black")

curve(gY(x, p = 0.4), add = TRUE, col = "black", lwd = 2)
```



Question 4

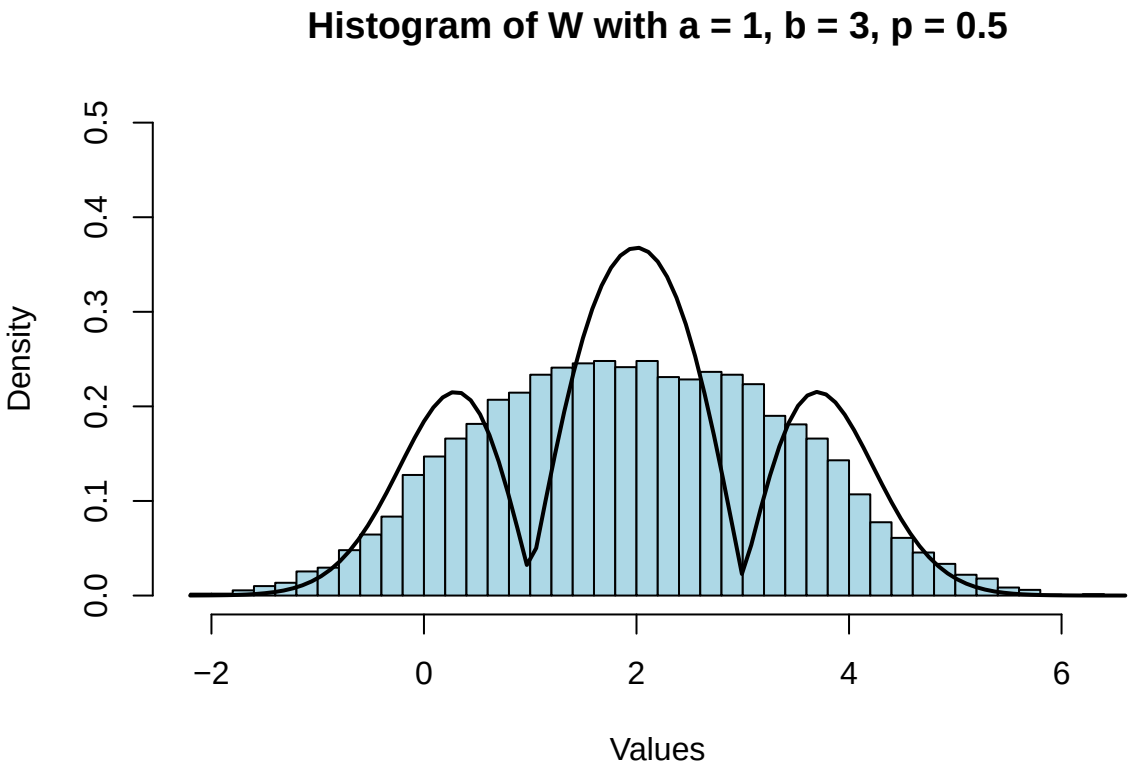
Given the equation

$$f_W(x) = ph(x-a) + (1-p)h(x-b)$$

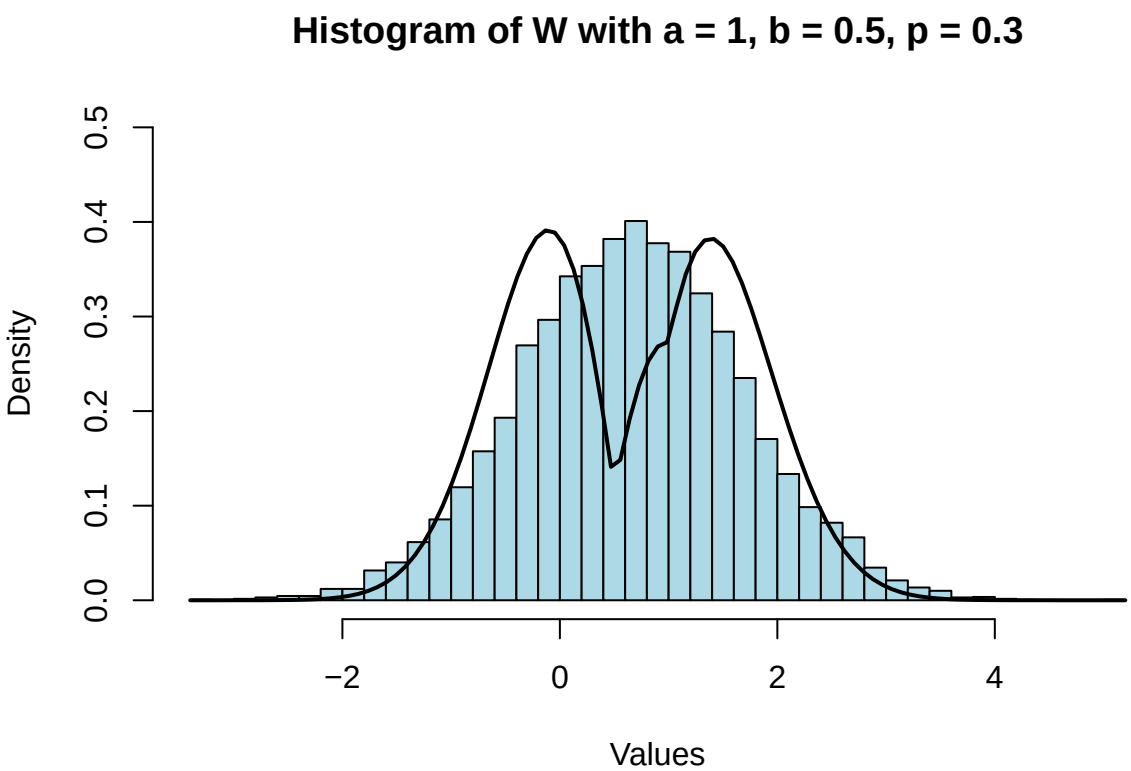
where $h(x) = |x|e^{-x^2}$, and a, b are real constants, and $p \in [0,1]$, we can write the function in R below:

```
# Code was partially generated by ChatGPT, but was edited/modified and understood before using it.
# Define the base PDF h(x)
h <- function(x) {
  abs(x) * exp(-x^2)
}

# Define the combined PDF f_W(x) for overlaying the histogram
fW <- function(x, a, b, p) {
  p * h(x - a) + (1 - p) * h(x - b)
}

# Define the function rmyW to generate random samples from f_W
rmyW <- function(n, a, b, p) {
  # Step 1: Determine which distribution to sample from
  from_a <- rbinom(n, 1, p)  # Vector of 0s and 1s, where 1 means sample shifted by a

  # Step 2: Generate samples from the base distribution h and shift them
  samples <- numeric(n)
  base_samples <- rnorm(n)  # Sample from standard normal as a base for |X|, where X ~ N(0,1)
  base_samples <- abs(base_samples) * sign(base_samples)  # Use |X| for the distribution h(x)
```

```r
  # Step 3: Apply the shifts
  samples[from_a == 1] <- base_samples[from_a == 1] + a
  samples[from_a == 0] <- base_samples[from_a == 0] + b

  return(samples)
}

# Generate 10,000 samples for the specified cases
set.seed(51940633)  # for reproducibility
sample1 <- rmyW(10000, a = 1, b = 3, p = 0.5)
sample2 <- rmyW(10000, a = 1, b = 0.5, p = 0.3)

# Plot histogram with PDF overlay for the first case
hist(sample1, probability = TRUE, breaks = 50,
     main = "Histogram of W with a = 1, b = 3, p = 0.5",
     xlab = "Values", col = "lightblue", border = "black", ylim=c(0,0.5))
curve(fW(x, a = 1, b = 3, p = 0.5), add = TRUE, col = "black", lwd = 2)
```




```r
# Plot histogram with PDF overlay for the second case
hist(sample2, probability = TRUE, breaks = 50,
     main = "Histogram of W with a = 1, b = 0.5, p = 0.3",
     xlab = "Values", col = "lightblue", border = "black",ylim=c(0,0.5))
curve(fW(x, a = 1, b = 0.5, p = 0.3), add = TRUE, col = "black", lwd = 2)
```



Question 5

(a)

```r
# Code was partially generated by ChatGPT, but was edited/modified and understood before using it.
rClinicalTrial <- function(n, mu, sigma, T) {
  # Generate recovery times for placebo group
  placeboTimes <- rlnorm(n, meanlog = mu[1], sdlog = sigma[1])
  placeboCensored <- ifelse(placeboTimes > T, 1, 0)
  placeboTimes <- ifelse(placeboTimes > T, T, placeboTimes)

  # Generate recovery times for treatment group
  treatmentTimes <- rlnorm(n, meanlog = mu[2], sdlog = sigma[2])
  treatmentCensored <- ifelse(treatmentTimes > T, 1, 0)
  treatmentTimes <- ifelse(treatmentTimes > T, T, treatmentTimes)

  # Combine results into a data frame
  data <- data.frame(
    group = rep(1:2, each = n),
    recovery_time = c(placeboTimes, treatmentTimes),
    censored = c(placeboCensored, treatmentCensored)
  )

  return(data)
}
```

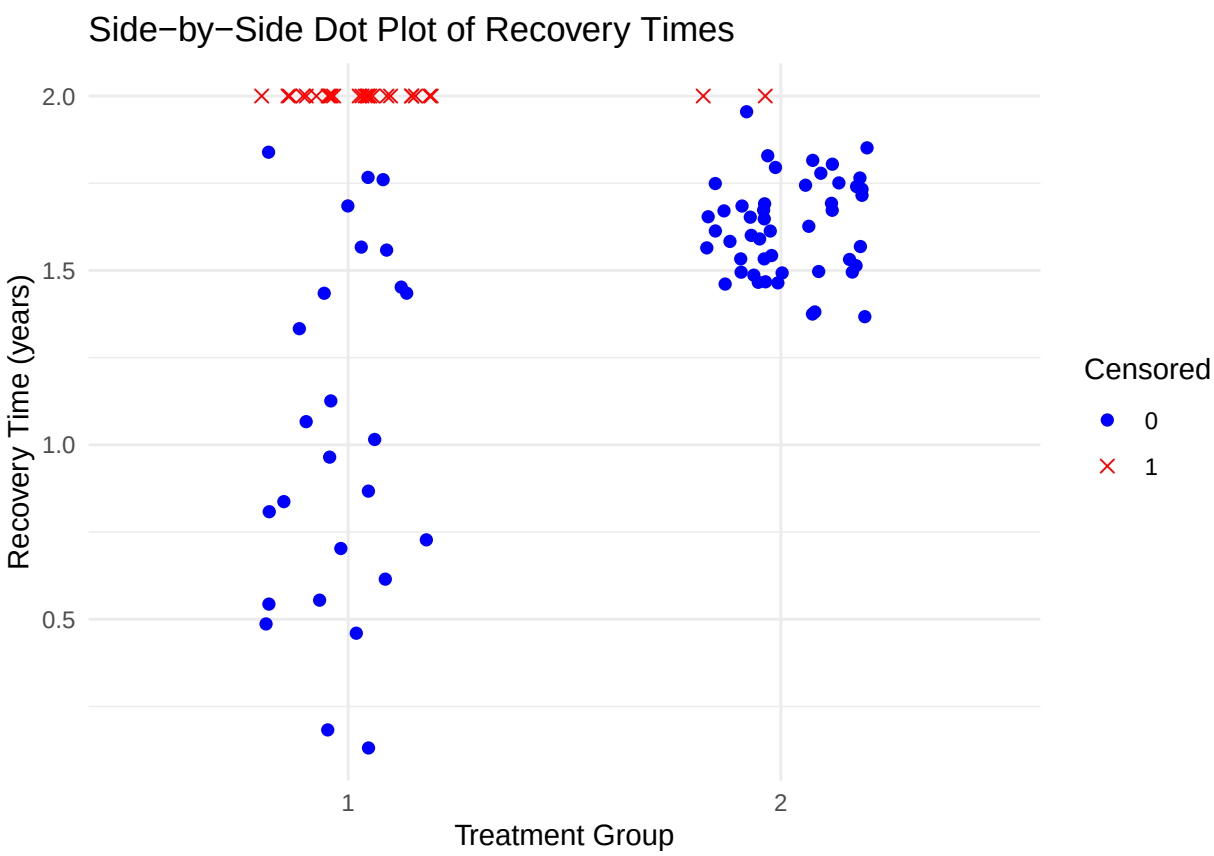
(b)

```r
# Set parameters
n <- 50
mu <- c(0.5, 0.5)
sigma <- c(1, 0.1)
T <- 2

# Simulate clinical trial data
set.seed(51940633)  # for reproducibility
clinical_data <- rClinicalTrial(n, mu, sigma, T)
head(clinical_data)  # Display the first few rows of data
```

```
##   group recovery_time censored
## 1     1     0.8371919        0
## 2     1     2.0000000        1
## 3     1     2.0000000        1
## 4     1     0.5435068        0
## 5     1     1.6848411        0
## 6     1     0.9647874        0
```

(c)

```r
# Load necessary library for plotting
library(ggplot2)

# Create a dot plot with different symbols for censored observations
ggplot(clinical_data, aes(x = factor(group), y = recovery_time,
  color = factor(censored))) +
  geom_jitter(width = 0.2, aes(shape = factor(censored)), size = 2) +
  scale_shape_manual(values = c(16, 4)) +  # 16: filled circle, 4: X mark for censored
  scale_color_manual(values = c("blue", "red")) +
  labs(
    x = "Treatment Group",
    y = "Recovery Time (years)",
    color = "Censored",
    shape = "Censored",
    title = "Side-by-Side Dot Plot of Recovery Times"
  ) +
  theme_minimal()
```



Question 6

(a)

```
# Code was partially generated by ChatGPT, but was edited/modified and understood before using it.
rClinicalTrial <- function(n, mu, sigma, T, alpha, beta) {
  # Generate recruitment times for all patients from Gamma distribution
  recruitment_times <- rgamma(2 * n, shape = alpha, scale = beta)
  censoring_times <- T - recruitment_times  # Time each patient has in the study

  # Generate recovery times for placebo group
  placebo_times <- rlnorm(n, meanlog = mu[1], sdlog = sigma[1])
  placebo_censored <- ifelse(placebo_times > censoring_times[1:n], 1, 0)
  placebo_times <- ifelse(placebo_times > censoring_times[1:n], censoring_times[1:n], placebo_times)

  # Generate recovery times for treatment group
  treatment_times <- rlnorm(n, meanlog = mu[2], sdlog = sigma[2])
  treatment_censored <- ifelse(treatment_times > censoring_times[(n + 1):(2 * n)], 1, 0)
  treatment_times <- ifelse(treatment_times > censoring_times[(n + 1):(2 * n)], censoring_times[(n + 1)

  # Combine results into a data frame
  data <- data.frame(
    group = rep(1:2, each = n),
    recovery_time = c(placebo_times, treatment_times),
    censored = c(placebo_censored, treatment_censored)
```

```r
  )

  return(data)
}
```

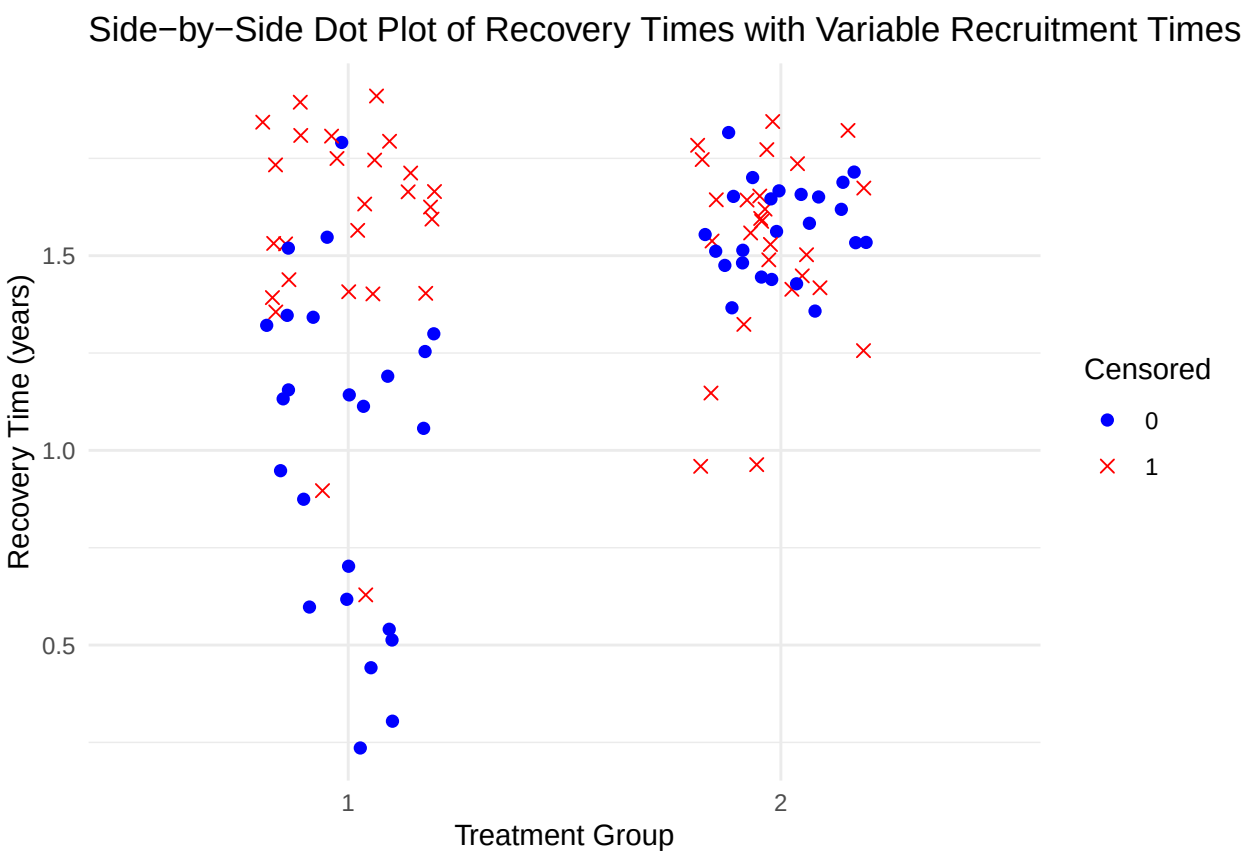
(b)

```r
# Set parameters
n <- 50
mu <- c(0.5, 0.5)
sigma <- c(1, 0.1)
T <- 2
alpha <- 2
beta <- 0.2

# Simulate clinical trial data
set.seed(51940633)  # for reproducibility
clinical_data <- rClinicalTrial(n, mu, sigma, T, alpha, beta)
head(clinical_data)  # Display the first few rows of data
```

```
##   group recovery_time censored
## 1     1     1.8430384        1
## 2     1     0.5977678        0
## 3     1     1.9102499        1
## 4     1     1.1132870        0
## 5     1     1.5304144        1
## 6     1     1.6646699        1
```

(c)

```r
# Load necessary library for plotting
library(ggplot2)

# Create a dot plot with different symbols for censored observations
ggplot(clinical_data, aes(x = factor(group), y = recovery_time,
  color = factor(censored))) +
  geom_jitter(width = 0.2, aes(shape = factor(censored)), size = 2) +
  scale_shape_manual(values = c(16, 4)) +  # 16: filled circle, 4: X mark for censored
  scale_color_manual(values = c("blue", "red")) +
  labs(
    x = "Treatment Group",
    y = "Recovery Time (years)",
    color = "Censored",
    shape = "Censored",
    title = "Side-by-Side Dot Plot of Recovery Times with Variable Recruitment Times"
  ) +
  theme_minimal()
```



Question 7

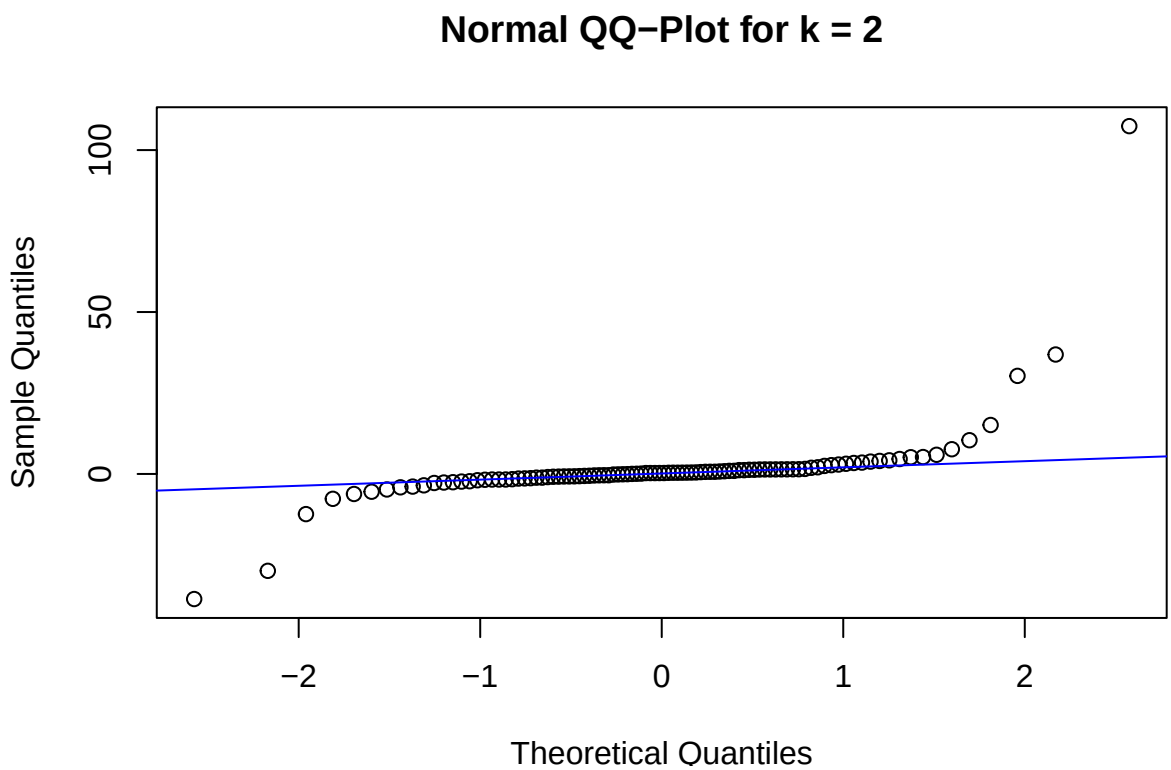
(a)

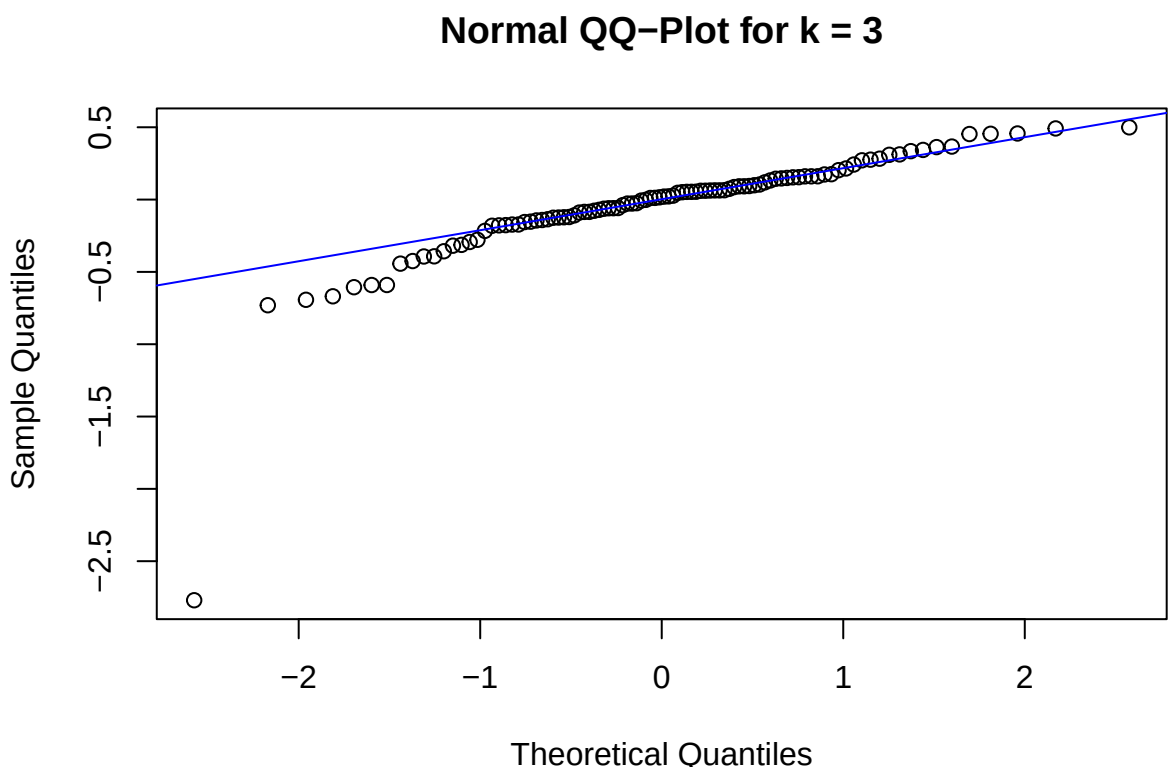
```
# Code was partially generated by ChatGPT, but was edited/modified and understood before using it.

rParetoX <- function(n, k) {
  # Generate Pareto distributed values (scale = 1, shape = k - 1)
  pareto_values <- (1 / runif(n))^(1 / (k - 1)) - 1

  # Assign random signs using rbinom
  signs <- ifelse(rbinom(n, 1, 0.5) == 0, -1, 1)

  # Apply signs to Pareto values
  simulated_values <- signs * pareto_values
  return(simulated_values)
}
```

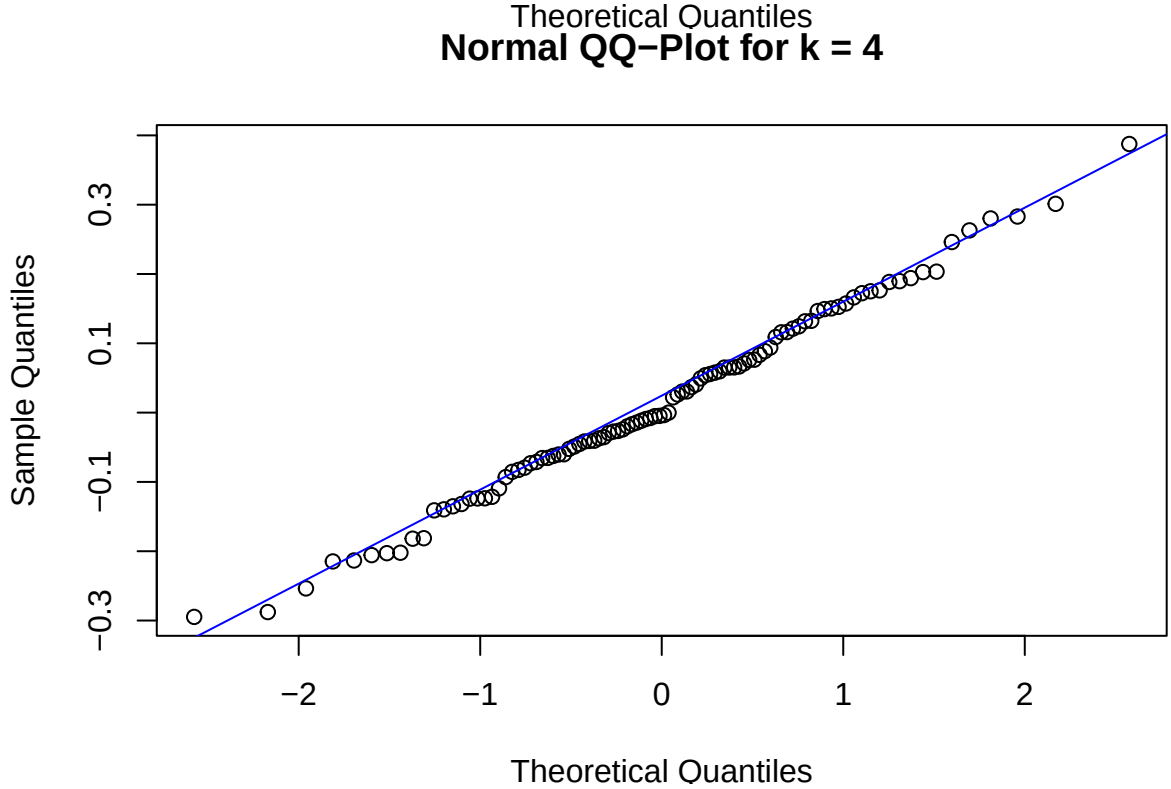
(b)

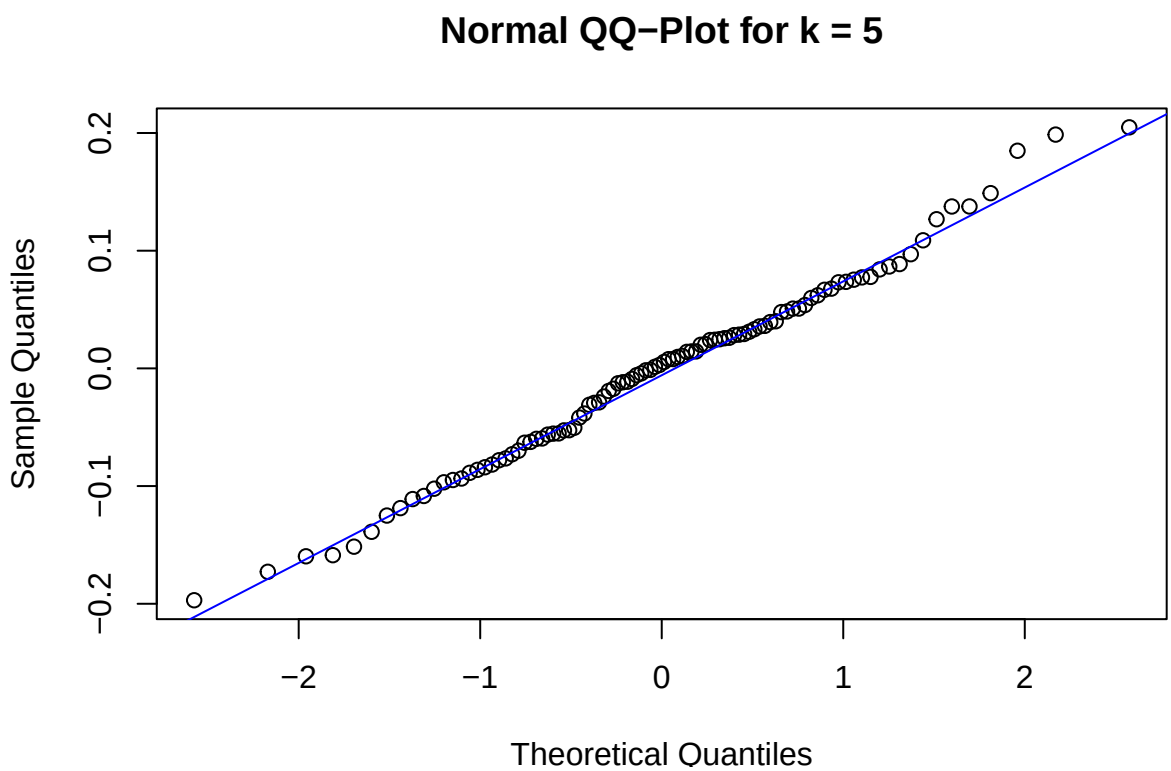
```
# Set parameters
sample_size <- 50
num_samples <- 100
k_values <- 2:5
```

```r
# Store the means for each k in a list
sample_means <- list()

# Loop over each value of k
for (k in k_values) {
  means <- numeric(num_samples)
  for (i in 1:num_samples) {
    # Generate a sample of size 50 for the given k
    sample <- rParetoX(sample_size, k)
    # Calculate the sample mean
    means[i] <- mean(sample)
  }
  # Store means for current k
  sample_means[[as.character(k)]] <- means
}

# Load required package for plotting
library(ggplot2)

# Plot QQ-plots for each k
for (k in k_values) {
  qqnorm(sample_means[[as.character(k)]], main = paste("Normal QQ-Plot for k =", k))
  qqline(sample_means[[as.character(k)]], col = "blue")
}
```




Normal QQ−Plot for k = 3
Sample Quantiles
Theoretical Quantiles
0.5
−0.5
−1.5
−2.5
−2
−1
0
1
2


Normal QQ−Plot for k = 4
Sample Quantiles
Theoretical Quantiles
0.3
0.1
−0.1
−0.3
−2
−1
0
1
2



(c) The Central Limit Theorem does not hold well for $k = 2$ because in the Normal QQ-Plot for $k = 2$ we can see that it is extremely heavy-tailed on the left. there is an outlier all the way at Sample Quantities = ~700, this also may imply that the Central Limit Theorem does not hold well for $k \leq 2$. For all the other k values, the plot seems to be normally distributed, which means that those k values are more stable.